Adam Edwards
aedwar14@my.athens.edu
ID: 00130309
Assignment 3

Intro to Assembly & Computer Organization

Chap 4.9.1

1. What will be the value in EDX after each of the lines marked (a) and (b) execute?

```
.data
one WORD 8002h
two WORD 4321h
.code
mov   edx,21348041h
movsx edx,one                    ; (a)
movsx edx,two                    ; (b)
```

- a. edx = FFFF8002h b. edx = 00004321h

2. What will be the value in EAX after the following lines execute?

```
mov eax,1002FFFFh
inc ax
```

- eax = 10020000h

4. What will be the value in EAX after the following lines execute?

```
mov eax,1002FFFFh
neg ax
```

- eax = 10020001h

6. What will be the value of EAX and the Sign flag after the following lines execute?

```
mov eax,5
sub eax,6
```

- eax = FFFFFFFFh, SF = 1 (bc result is negative)

8. What value will RAX contain after the following instruction executes?

```
mov rax,44445555h
```

- rax = 0000000044445555h

10. What value will EAX contain after the following instructions execute?

```
.data
dVal DWORD 12345678h
.code
mov ax,3
mov WORD PTR dVal+2,ax
mov eax,dVal
```

- eax = 00035678h

16. For each of the following statements, state whether or not the instruction is valid:

```
a. mov ax,var1
b. mov ax,var2
c. mov eax,var3
d. mov var2,var3
e. movzx ax,var2
f. movzx var2,al
g. mov ds,ax
h. mov ds,1000h
```

- a. not valid b. valid c. not valid d. not valid e. not valid f. not valid g. valid h. not valid

18. What will be the value of the destination operand after each of the following instructions execute in sequence?

```
mov ax,var2                    ; a.
mov ax,[var2+4]                ; b.
mov ax,var3                    ; c.
mov ax,[var3-2]                ; d.
```

- a. 1000h b. 3000h c. FFF0h d. 4000h

Chap 4.9.2

1. Write a sequence of MOV instructions that will exchange the upper and lower words in a doubleword variable named **three**.

- Code:

  mov ax, word ptr three
  mov bx, word ptr three+2
  mov three, bx
  mov word ptr three+2, ax

8. Write a loop that iterates through a doubleword array and calculates the sum of its elements using a scale factor with indexed addressing.

- Code:

  mov al, 80h
  add al, 80h

Chap 4.10

2. Write a program with a loop and indexed addressing that exchanges every pair of values in an array with an even number of elements. Therefore, item i will exchange with item i+1, and item 1+2 will exchange with item i+3, and so on.

Chap 5.8.1

1. Which instruction pushes all of the 32-bit general-purpose registers on the stack?

- pusha

2. Which instruction pushes the 32-bit EFLAGS register on the stack?

- pushf

5. *Challenge:* Suppose MASM contained no PUSH instruction. Write a sequence of two other instructions that would accomplish the same as push eax.

- Code:

  sub esp, 4
  mov [esp] ,eax

18. Which statement is true about what will happen when the example code runs?

```
 1: main PROC
 2:      mov eax,40
 3:      push offset Here
 4:      jmp Ex4Sub
 5:   Here:
 6:      mov eax,30
 7:      INVOKE ExitProcess,0
 8: main ENDP
 9:
10: Ex4Sub PROC
11:      ret
12: Ex4Sub ENDP
```

- c. EAX will equal 30 on line 6

20. What values will be written to the array when the following code executes?

```
.data
array DWORD 4 DUP(0)
.code
main PROC
     mov  eax,10
     mov  esi,0
     call proc_1
     add  esi,4
     add  eax,10
     mov  array[esi],eax
     INVOKE ExitProcess,0
main ENDP
proc_1 PROC
     call proc_2
     add  esi,4
     add  eax,10
     mov  array[esi],eax
     ret
proc_1 ENDP
proc_2 PROC
     call proc_3
     add  esi,4
     add  eax,10
     mov  array[esi],eax
     ret
proc_2 ENDP
proc_3 PROC
     mov  array[esi],eax
     ret
proc_3 ENDP
```

- The array will contain 10, 20, 30, 40

Chap 5.8.2

3. Functions in high-level languages often declare local variables just below the return address on the stack. Write an instruction that you could put at the beginning of an assembly language subroutine that would reserve space for two integer doubleword variables. Then, assign the values 1000h and 2000h to the two local variables.

- Code:

```
Sub esp, 8              ; space for two variables
mov [esp] ,1000h
mov [esp+4] ,2000h
```

5. Write a sequence of statements that display a subroutine's return address. Be sure that whatever modifications you make to the stack do not prevent the subroutine from returning to its caller.

- Code:

  mov eax. [esp]
  call WriteHex

Chap 5.9

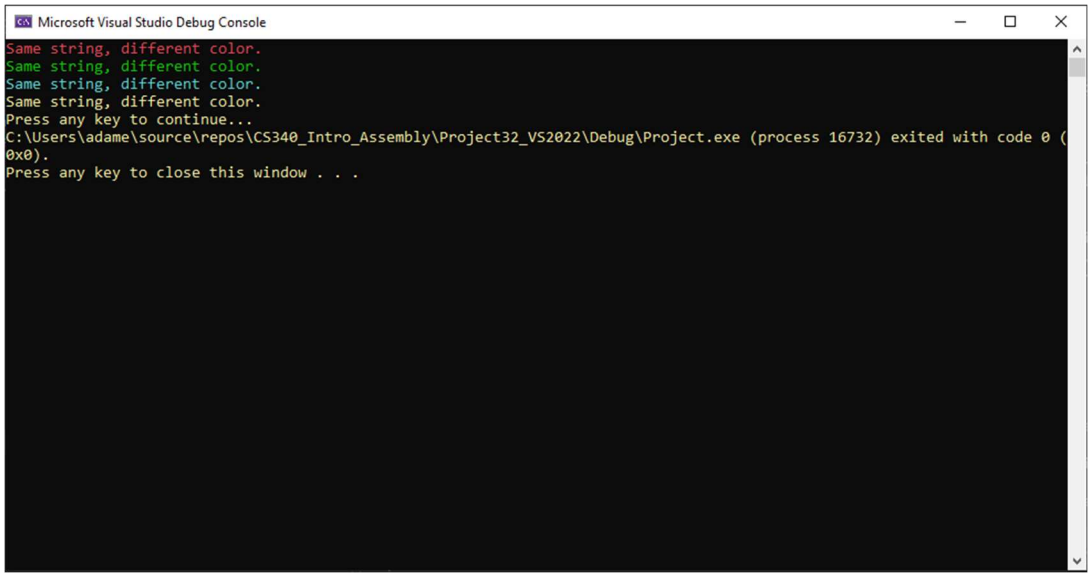
1. Write a program that displays the same string in four different colors, using a loop. Call the SetTextColor procedure from the book's link library. Any colors may be chosen, but you may find it easiest to change the foreground color.

Input:

Ch05_9-01.asm

```
;-----------------------------------------------------------
; Program: Chapter 5.9 #1 - Print Same Message in 4 Colors
; Purpose:
;   - Prints the same message 4 times.
;   - Each time it changes the text color before printing.
;   - Uses a loop and the Irvine32 SetTextColor procedure.
;-----------------------------------------------------------

.386
.model flat, stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD

include Irvine32.inc

.data
message BYTE "Same string, different color.", 0

; A list (array) of 4 color values we want to use.
; These color names come from Irvine32.inc.
colorList BYTE lightRed, lightGreen, lightCyan, yellow

.code
main PROC

    ; ESI will point at the current color in colorList
    mov esi, OFFSET colorList

    ; ECX is the loop counter (how many times to repeat)
    mov ecx, LENGTHOF colorList      ; = 4

PrintMessageInNextColor:

    ; Load the next color (a BYTE) from memory into EAX
    ; movzx = "move with zero-extend" so EAX becomes 0..255 cleanly
    movzx eax, BYTE PTR [esi]

    ; Set the console text color using Irvine32
    call SetTextColor

    ; Print the message
    mov edx, OFFSET message
    call WriteString
    call Crlf

    ; Move to the next color in the list
    inc esi

    ; LOOP does: ECX = ECX - 1, and if ECX != 0, jump back to the label
    loop PrintMessageInNextColor

    ; Pause so you can see output (good for screenshots)
    call WaitMsg

    invoke ExitProcess, 0
main ENDP
```

Output:


Microsoft Visual Studio Debug Console
Same string, different color.
Same string, different color.
Same string, different color.
Same string, different color.
Press any key to continue...
C:\Users\adame\source\repos\CS340_Intro_Assembly\Project32_VS2022\Debug\Project.exe (process 16732) exited with code 0 (
0x0).
Press any key to close this window . . .

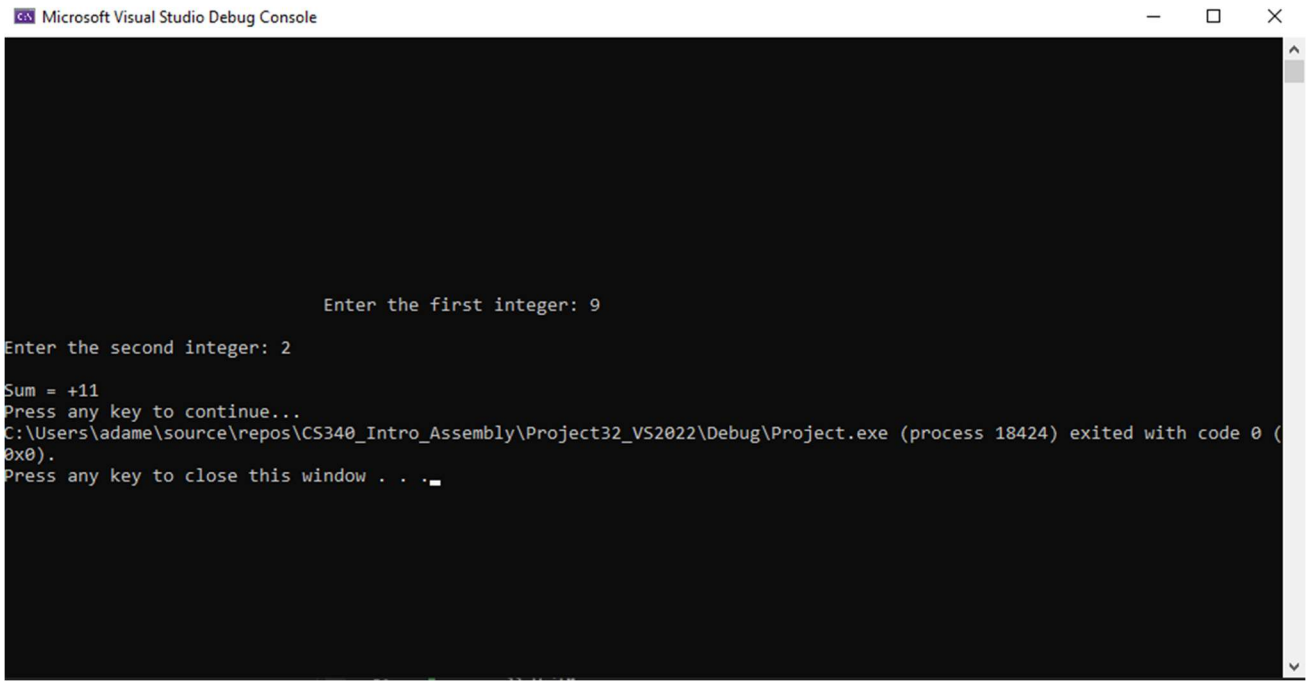
3. Write a program that clears the screen, locates the cursor near the middle of the screen, prompts the user for two integers, adds the integers, and displays their sum.

Input:

CH05_9-03.asm

```
;-----------------------------------------------------------
; Program: Chapter 5.9 #3 - Add Two Integers
; Purpose:
;   - Clears the screen
;   - Moves the cursor near the middle
;   - Prompts user for two integers
;   - Adds them and displays the sum
;-----------------------------------------------------------

.386
.model flat, stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD

include Irvine32.inc

.data
prompt1 BYTE "Enter the first integer: ",0
prompt2 BYTE "Enter the second integer: ",0
resultMsg BYTE "Sum = ",0

.code
main PROC
    ; Clear the console screen
    call Clrscr

    ; Move cursor near middle of screen (row, col)
    ; 25-row console: middle is around row 12
    ; 80-col console: middle is around col 30
    mov dh, 12          ; row
    mov dl, 30          ; column
    call Gotoxy

    ; Ask for first integer
    mov edx, OFFSET prompt1
    call WriteString
    call ReadInt        ; result goes into EAX
    mov ebx, eax        ; save first integer in EBX

    call Crlf

    ; Ask for second integer
    mov edx, OFFSET prompt2
    call WriteString
    call ReadInt        ; EAX = second integer

    ; Add them: EAX = second + first
    add eax, ebx

    call Crlf
    mov edx, OFFSET resultMsg
    call WriteString
    call WriteInt
    call Crlf

    call WaitMsg
    invoke ExitProcess, 0
main ENDP

END main
```

Output:


Microsoft Visual Studio Debug Console
Enter the first integer: 9
Enter the second integer: 2
Sum = +11
Press any key to continue...
C:\Users\adame\source\repos\CS340_Intro_Assembly\Project32_VS2022\Debug\Project.exe (process 18424) exited with code 0 (
0x0).
Press any key to close this window . . .